

- Hacen que sea más fácil reutilizar buenos diseños y arquitecturas.
- Son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software; es un concepto general para resolver un problema particular.

Ayudan a:

- Encontrar objetos: identificar abstracciones (clases como estrategia/estado) no tan obvias y los objetos que las expresan; implementadas mediante clases de fabricación pura (clases que no representan objetos reales).
- Determinar la granularidad de los objetos: qué tanto va a hacer una clase; cuánto comportamiento tienen las clases; cuántos métodos implementamos en cada clase.
- Especificar interfaces de los objetos: encontrar interfaces que van a contener el conjunto de peticiones que pueden realizar los clientes, y que éstos no tengan que preocuparse por las distintas clases concretas que se van a encargar de esas peticiones.
- Especificar la implementación de un objeto: código que va dentro de los métodos; hay ciertos patrones que ayudan a saber qué métodos/algoritmos debemos usar.
- Favorecer la reutilización: herencia de interfaces (describe cuando un objeto se puede usar en el lugar de otro), herencia de clases (definir una implementación en términos de otra; reutilización de caja blanca (las interioridades de las clases padres suelen hacerse visibles a las subclases rompiendo el encapsulamiento)) vs composición de objetos (la nueva funcionalidad se obtiene ensamblando o componiendo objetos; reutilización de caja negra (detalles internos de los objetos no son visibles)). En la composición hacemos uso de la delegación (aprovechar el código que está escrito en otra clase, sin tener que romper el encapsulamiento y sin tener que escribir el código en la clase contexto, invoca al método para que otra clase lo resuelva).
- Diseñar para el cambio: anticipar nuevos requisitos y cambios; analizar cómo puede necesitar cambiar el sistema. Diseñar el software de forma que sea flexible, robusto y fácil de evolucionar, ya que los requerimientos cambian. Los patrones nos ayudan a estar preparados para que haya cambios en el software, y para que los podamos implementar al menor costo posible.

De Creación: Abstraen el proceso de creación y ocultan los detalles de cómo los objetos son creados, mejorando y flexibilizando tal proceso a través de la distribución de responsabilidades. Ayudan a hacer a un sistema independiente de cómo se crean, se componen y se representan sus objetos.

Estructurales: Determinan cómo combinar objetos y clases para definir estructuras complejas. Explican cómo ensamblar objetos y clases en estructuras más grandes (cómo se combinan clases y objetos para formar estructuras).

De Comportamiento: Manejan la comunicación entre objetos del sistema y el flujo de información entre ellos. Se encargan de delegar y distribuir las responsabilidades entre los objetos.

- ❖ **Singleton**: patrón de creación que garantiza que una clase sólo tenga una instancia, y proporciona un acceso global a ella.
- ❖ **Adapter**: patrón estructural que funciona como intermediario entre dos clases que no pueden comunicarse. Convierte la interfaz de una clase en otra que es la que esperan los clientes. Cuando existen clases incompatibles, adapta una interfaz para que pueda ser utilizada por otra clase.
- ❖ **State**: patrón de comportamiento que encapsula los estados de un objeto, para que éste pueda cambiar su comportamiento cuando cambia su estado. Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- ❖ **Strategy**: patrón de comportamiento que define una familia de algoritmos, encapsula cada uno y permite intercambiarlos. Permite que un algoritmo varíe independientemente de los clientes que lo usan. Encapsula un algoritmo en un objeto, facilitando especificar y cambiar el algoritmo que usa un objeto.
- ❖ **Iterator**: patrón de comportamiento que proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación interna.
- ❖ **Observer**: patrón de comportamiento que define y mantiene una dependencia entre objetos, de forma que, cuando un objeto cambie de estado se notifique y se actualicen todos los objetos que dependen de él.

- ❖ **Template Method:** patrón de comportamiento que define el esqueleto de un algoritmo en la superclase, permitiendo que las subclases sobrescriban pasos del algoritmo sin cambiar su estructura.

PRINCIPIOS DE DISEÑO

- Son una guía de las buenas prácticas que se deben seguir en el diseño de software, y ayudan a generar un software mantenible, extensible y flexible.
- Foco es mantener alta la cohesión, bajo el acoplamiento, facilitar los cambios y prevenir errores.
- Son importantes porque nos ayudan a determinar si estamos generando un software de calidad o no.

Características del buen diseño:

- Reutilización de código: reducir costos del desarrollo; utilizar código existente en nuevos proyectos.
- Extensibilidad: cambio es la única constante; prever posibles cambios cuando se diseña la arquitectura de las apps.
- Identificación y tratamiento de excepciones: que el usuario frente a un error sepa cómo tiene que seguir para recuperarse del mismo.
- Prevención y tolerancia de defectos: software contemple si algo sale de lo normal.
- Independencia de componentes: bajo acoplamiento y alta cohesión.

SOLID

Responsabilidad → obligación del objeto en cuanto a su comportamiento, ≠ método.

- ❖ **Responsabilidad única:** Clase debe tener sólo una razón para cambiar. Que cada clase sea responsable de una única parte de la funcionalidad del sw. Y que esa responsabilidad esté totalmente encapsulada por la clase. (Alta cohesión y facilita mantenimiento).

Pregunta de validación → la clase puede resolver el comportamiento sola?

1. State: Estados particulares en clases separadas. Cada clase de estado concreto define el comportamiento en un estado específico.
2. Strategy: Cada estrategia tiene una única responsabilidad, que es implementar un algoritmo específico.
3. Iterator: Se extraen los algoritmos de recorrido del cliente y se los coloca en clases independientes que se encarguen de gestionar la iteración y acceso a los elementos.
Separa la lógica de la iteración de la colección en sí misma. Le saca la responsabilidad de la clase de saber cómo iterarse y se lo delega al iterador.
Puede limpiar el código del cliente y las colecciones, extrayendo algoritmos de recorrido voluminosos en clases separadas.
4. Adapter: Se crea una clase que se encargue de la conversión, en lugar de que esta responsabilidad caiga sobre el cliente.

- ❖ **Abierto-Cerrado:** Clases deben estar abiertas para extensión y cerradas para modificación. Ante la necesidad de un cambio, no debería cambiar el sw existente, sino agregar algo nuevo que trabaje con lo anterior sin modificar su funcionamiento. Los nuevos comportamientos se agregan en nuevas clases. (Herencia / Realización).

1. State: Se pueden agregar nuevos estados sin cambiar los preexistentes o el contexto.
2. Strategy: Se pueden agregar nuevas estrategias sin cambiar las preexistentes o el contexto.
Se pueden añadir nuevas estrategias sin alterar el código de las clases que utilizan la estrategia. Esto se hace con nuevas clases que implementen la interfaz, ampliando la funcionalidad.
3. Iterator: Agregar nuevos tipos de iteradores para colecciones nuevas sin afectar a las existentes.
4. Observer: Los sujetos y observadores pueden cambiar y evolucionar independientemente, sin requerir modificaciones en el otro. Se pueden agregar nuevas clases suscriptoras sin cambiar el código del publicador (y viceversa).
5. Adapter: Se pueden introducir nuevos tipos de adaptadores sin romper el código existente.
Se agrega una clase adaptadora que extiende el comportamiento de la clase cliente, con el objetivo de relacionarse con un adaptable para que puedan comunicarse, ya que antes eran incompatibles. La clase cliente se mantiene intacta.

❖ **Sustitución de Liskov:** Una clase hija debe poder ser usada en el lugar de la clase base sin cambiar el comportamiento del programa. Pasar los objetos de la subclase en el lugar de los objetos de la clase padre sin romper el código existente. Evalúa si la herencia está bien diseñada. El subtipo debe comportarse como el supertipo al extender el comportamiento de este. No es que no se pueda comportar diferente, pero si de alguna manera, limitamos la interacción a los métodos del padre, vamos a obtener el comportamiento esperado.

1. State: Los concretos se pueden intercambiar con el abstracto, ya que todos son capaces de responder a sus métodos.
Los estados heredan de una clase base común, entonces cada estado deber ser un reemplazo válido (conceptualmente) de la clase base.
2. Template Method: Analiza si está bien armada la herencia. El método base proporciona una plantilla que puede ser extendida por las subclases sin modificar su estructura fundamental. Las subclases añaden o modifican pasos específicos, pero no alteran el flujo general del proceso.

❖ **Segregación de Interfaces:** No forzar a los clientes a depender de métodos que no utilizan, ni forzar a las clases concretas a implementar comportamientos que no necesitan. Se deben desintegrar las interfaces gruesas hasta crear otras más detalladas y específicas.

A través de la interfaz, las clases concretas implementan el comportamiento declarado en la interfaz; por ende, están obligadas a implementar todos los métodos declarados, los necesiten o no. No debemos obligar a los clientes a vincularse con interfaces muy complejas. Como las clases pueden implementar más de una interfaz (contrario a la herencia), se busca separar en más interfaces con un mayor nivel de cohesión, donde las clases sólo implementen las que necesiten.

1. Observer: 2 interfaces; pongo el comportamiento que tiene que ver con los sujetos en la ISujeto para que los sujetos concretos lo implementen; y con el observador igual. Por ende, al estar divididas, no obligan a implementar los métodos a quienes no los necesitan.
2. Iterator: Se proporciona una interfaz simple y específica para recorrer colecciones, sin agregar métodos innecesarios.
“IAgregado” – crearIterator()
“IIterator” – comprobarFiltros(), elementoActual(), primero(), siguiente(), haFinalizado()

❖ **Inversión de dependencia:** Al desarrollar sw muchas veces se desarrollan primero los componentes de bajo nivel y después los de alto nivel. Entonces las clases de alto nivel terminan dependiendo de las de bajo nivel. No es ideal que las capas de nivel superior dependan de detalles de implementación de las capas de nivel inferior. Por eso se busca “invertir” el orden natural de la dependencia.

Relación natural de la herencia es que los hijos llaman al padre. Las clases de alto nivel no deben depender de clases de bajo nivel, ambas deben relacionarse con abstracciones y no con concreciones.

Se busca evitar que los módulos de alto nivel dependan de los de bajo nivel, sino desacoplar poniendo una interfaz en el medio, entonces los módulos de bajo nivel proveen los servicios que requiere el módulo de alto nivel, el módulo de alto nivel necesita un servicio y el de bajo nivel se lo provee, pero a través de una interfaz.

1. Template Method: Se invierte el orden natural de la dependencia a través del método plantilla, en el cual el padre invoca comportamientos del hijo.
Método del padre, se invierte la herencia, porque en el método plantilla llama a los métodos que tiene definidos en la clase padre y también llama a los métodos específicos de la clase hija sobre la cual se está instanciando. Se permite que la clase plantilla arme una plantilla completa, con invocaciones a clases concretas.

OTROS

- ❖ **Programar hacia la interfaz no hacia la implementación:** el objetivo es hacer que las clases dependan de abstracciones (interfaces o clases abstractas) en lugar de concreciones (implementaciones específicas); para así lograr un mejor desacoplamiento entre los componentes y reducir las dependencias de implementación entre los subsistemas. (observer-strategy)
- ❖ **Favorecer la composición por sobre la herencia:** la herencia genera fuerte acoplamiento entre las clases hijas y el padre, y rompe el encapsulamiento, ya que ellas acceden a los atributos del padre, y una modificación en el padre hace que también cambien las hijas. Se propone usar composición, que nos permite reutilizar comportamiento a través de la delegación, manteniendo un acoplamiento menor, y sin romper el encapsulamiento. A través de la delegación usamos polimorfismo, dado que

las clases están encapsuladas y pueden tener el mismo método, misma signatura y los objetos se van a comportar de acuerdo a su clase, no metiéndome en la estructura interna del código. (strategy)

- ❖ **Encapsular lo que varía:** busca separar el código de las partes que son propensas a cambiar y encapsularlas en otros objetos, para que cada vez que haya que hacer un cambio, el impacto esté localizado en una pequeña y única parte del sistema.
(state-strategy)
- ❖ **No te repitas:** busca que una funcionalidad del sistema esté representada por una única porción de código.
(state-strategy)
- ❖ **Decir, no preguntar:** distribución de responsabilidades, entonces un objeto le pide a otro que haga una determinada cosa y le devuelve un resultado. Busca llevar el código hacia donde se encuentran los datos que utiliza. Busca que no existan solamente clases con métodos get y set. Si un objeto tiene los datos necesarios para realizar una función, mejor que él sea el responsable. No usar los objetos para pedirles cosas, sino que hay que decirles a los objetos que hagan cosas. Establece que los objetos deben pedir a otros que hagan algo por ellos, en lugar de preguntarles por datos o tomar decisiones ellos mismos.
(state-strategy)

ARQUITECTURA

Conjunto de decisiones significativas que tomamos sobre cómo resolver los RNF, teniendo en cuenta el contexto (reglas de negocio).

Se diseña porque en ésta es donde se ejecuta el sw; es la base en donde se ejecutan las distintas funcionalidades del sistema.

PROCESO DE DISEÑO ARQUITECTÓNICO

Es iterativo e incremental; porque no es que diseñas una vez la arquitectura y después no se modifica más, sino que lo vas actualizando a medida que vas incorporando nuevos conocimientos sobre el dominio y que vas validando lo que hiciste.

1) **Determinar los requerimientos arquitectónicos**

1ro Identificar los requerimientos arquitectónicos: Se toma como entrada los reqs funcionales y los de los involucrados. Se determinan los reqs arquitectónicos, y se produce como salida un doc que contiene los reqs arquitectónicos de la app.

2do Priorizar requerimientos: Alta (la app debe soportarlo, conducen el diseño de arquitectura, tienen que estar disponibles desde la primera iteración), Media (debe ser soportado en alguna etapa, pero no necesariamente en la primera), Baja (parte de la lista de deseos, no conducen el diseño, pueden ser postergados para las últimas iteraciones).

2) **Diseño arquitectónico**

Proceso que toma el modelo de análisis y los RNF (arquitectónicos), y los aplica sobre una tecnología específica.

Incluye la definición de una estructura de componentes junto con las responsabilidades que tendrán cada uno de ellos.

a) **Elegir framework de arquitectura**

Elección de los patrones que voy a aplicar en el producto de sw.

Ver de todas las soluciones arquitectónicas del mercado, cuál nos servirá a nosotros para implementar, dado los RNF y las características del producto.

Se elige un framework que puede constar de uno o más patrones arquitectónicos para definir y empezar a diseñar la arquitectura.

b) **Distribuir componentes**

Se deben definir los componentes principales que comprenderán el diseño.

Vamos a distribuir los componentes teniendo en cuenta qué componentes se van a utilizar, las dependencias entre ellos, la info que van a manejar, etc.

Salidas: Documento de arquitectura y Vistas arquitectónicas.

Documento de arquitectura

Sirve para documentar el producto de sw. Es un lugar donde voy poniendo las decisiones que tomo para construir la arquitectura de sw y el por qué, para dejar un registro de esas decisiones.

- Documentar sirve para → la comunicación entre los distintos participantes.
→ hacer análisis y determinar si se cumplen los reqs críticos.

Vistas de la arquitectura

Utilizadas para modelar/reflejar la arquitectura.

Pensadas para un involucrado en particular y sus intereses.

Modelan un punto de vista, que es una descripción teórica de lo que se va a mostrar en esa vista.

4+1 4 vistas unidas por la Vista de Casos de Uso

- **Vista de CU (funcionalidad)**
 - > Contiene los CU significativos para la arquitectura (capturan los requisitos para la arquitectura). Representa los RF clave.
 - > Captura y visualiza los principales CU, mostrando cómo los objetos y procesos interactúan para cumplir con los requisitos funcionales.
 - > A partir de esta vista se construyen las demás, respetando la característica del PUD “conducido por casos de uso”.
 - > Diagramas: estática (casos de uso), dinámica (secuencia o comunicación).
- **Vista de Diseño (lógica)**
 - > Describe los elementos significativos de la arqu y las relaciones entre ellos.
 - > Describe cómo será provista la funcionalidad del sistema (qué clases, objetos o componentes voy a utilizar para darle soporte a la funcionalidad).
 - > Captura la estructura de la aplicación.
 - > Muestra como las clases, subsistemas e interfaces trabajan juntos para implementar los casos de uso.
 - > Diagramas: estática (componentes), dinámica (secuencia).
- **Vista de Proceso:** Describe la concurrencia y los elementos de comunicación de una arquitectura. Enfocarse en los aspectos dinámicos del sistema, como procesos concurrentes y su comunicación. Se muestran los procesos que hay en el sistema, y la forma en la que se comunican estos procesos.
 - > Diagramas: estática (actividad), dinámica (secuencia).
- **Vista de Despliegue (física)**
 - > Describe cómo los principales procesos y componentes se asignan al hardware de aplicaciones.
 - > Describe cómo los componentes se distribuyen en la infraestructura física.
 - > Muestra cómo el sw será alojado en los diferentes componentes de hw.
 - > Diagramas: estática (despliegue), dinámica (secuencia).
- **Vista de Implementación (componentes)**
 - > Captura la organización interna de los componentes de código; normalmente se mantiene en un entorno de desarrollo o herramienta de gestión de la configuración.
 - > Cómo se organizan y gestionan los componentes del sw.
 - > Describe las dependencias de los módulos de implementación (código fuente, bibliotecas, componentes de terceros).
 - > Representar la arquitectura desde el punto de vista del programador, incluyendo la administración de los artefactos de sw.
 - > Mostrar cómo está dividido el sistema software (sw+hw) en componentes y las dependencias que hay entre esos componentes.
 - > Diagramas: estática (paquetes/componentes), dinámica (secuencia).

3) Validación

Es probar la arquitectura, comúnmente recorriendo el diseño contra los reqs existentes y los que se pueden predecir en el futuro.

Sirve para mostrar que la propuesta arquitectónica realmente resuelve los requerimientos (arquitectura cumpla su propósito).

Técnicas de validación

Objetivo: identificar potenciales defectos y debilidades en el diseño para que puedan ser mejorados antes de la fase de implementación.

✓ *Uso de escenarios*

Consiste en un testeo manual de la arquitectura usando casos de prueba.

Cuentan con una serie de casos de prueba que validarán el sistema en cada uno de los contextos de los casos de prueba.

✓ *Prototipos*

Primera aproximación del sistema funcionando.

Ayuda a brindarle al usuario una primera impresión del sistema.

Sirve para probar las distintas funcionalidades que tiene un sistema.

Tiene la capacidad de evaluar los reqs en forma detallada.

EXPERIENCIA DE USUARIO – UX

Significa hacer que una tecnología sea amigable, satisfactoria, fácil de usar y útil.

➤ UX → cómo funciona el producto; lo fácil o difícil que es de usar.

➤ UI → cómo se ve el producto; si se ve amigable o no.

Características/atributos del producto

- Accesibilidad: Qué tan posible es que el producto sea usado por la mayor cantidad de personas posibles sin problema.
- Utilidad: Verifica que se cumplan los requerimientos.
- Usabilidad: Mide la UX, cómo se siente el usuario al utilizar el sistema, y si puede cumplir sus objetivos de manera eficiente.

Atributos:

- Aprendizaje: cuánto tiempo tarda el usuario en aprender a usar el sistema y ser productivo.
- Velocidad de funcionamiento: cómo responde el sistema a acciones de trabajo que realiza el usuario sobre él.
- Robustez: qué tolerancia tiene el sistema a los errores del usuario.
- Recuperación: cómo se recupera el sistema a los errores del usuario.
- Adaptación: qué tan atado está el sistema a una forma de trabajar o modelo de trabajo; si el modelo de trabajo cambia (presencial a virtual), tengo que cambiar el sistema o se puede adaptar a ese modelo.
- Arquitectura de la información: Cómo los usuarios buscan, encuentran y entienden la información. Es el arte, ciencia y práctica de diseñar espacios interactivos y comprensibles.
- Modelos mentales: Comprender cómo y con qué fines los usuarios utilizarán el producto, para así diseñar una interfaz adaptada al modelo mental del usuario.

8 REGLAS DE ORO/SCHNEIDERMAN + Heurísticas Nielsen

1. **Buscar siempre la coherencia**: Mantén un diseño consistente en todo el sistema; mismos estilos de presentación (que la cruz esté arriba a la derecha).
 - Mantener la consistencia y estándares: (botones de confirmación verdes, que sea así para todo el sistema).
2. **Permitir el uso de atajos**: Facilita el acceso rápido a funciones comunes; para usuarios más avanzados.
 - Flexibilidad y eficiencia en el uso.
3. **Dar retroalimentación de información**: Por cada acción que realiza el usuario, debería existir una respuesta que sea visible (ruedita de carga).
 - Visibilidad del estado del sistema: mostrarle al usuario en qué estado se encuentra el sistema.

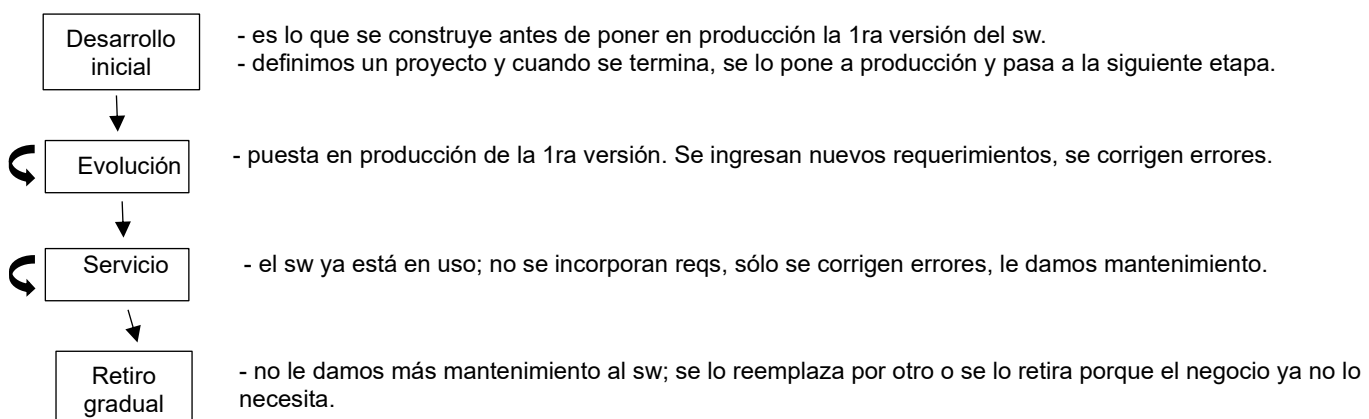
4. **Diseñar diálogos que tengan un fin:** Cada interacción debe tener un propósito claro. No incluir cosas que no son necesarias y que no agregan valor al usuario (\$ boletos para un origen y destino seleccionado).
 - Diseño estético y minimalista: visualizar la info justa y necesaria; no agregar info o datos de más que a veces son inútiles y confunden al usuario.
 5. **Permitir el manejo simple de errores:** Ofrecer soluciones claras y simples para cuando ocurran errores. Mostrarle al usuario cómo recuperarse de un error.
 - Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de errores: mostrar mensajes de error que sean claros y que ayuden a los usuarios a entender cuál es el problema y qué es lo que tienen que hacer para recuperarse de ese error.
 6. **Permitir deshacer las acciones con facilidad:** Dejar que el usuario revierta acciones fácilmente; permitirle al usuario volver atrás y arrepentirse de algo que hizo (ctrl z).
 - Libertad y control por parte del usuario.
 7. **Permitir que el centro de control sea interno:** El usuario debe sentir que controla la interacción. Hacerle creer al usuario que él tiene el control.
 - Visibilidad del estado del sistema.
 - Libertad y control por parte del usuario.
 - Prevenir errores: tratar de frenar al usuario antes de que cometa un error (cuadros de confirmación antes de realizar una acción destructiva).
 8. **Reducir la carga de memoria inmediata:** Evitar que el usuario tenga que recordar mucha información mostrando datos relevantes en la pantalla. No hacerle recordar al usuario lo que vio en la pantalla anterior.
 - Reconocer antes que recordar: evitar mucho texto, utilizar imágenes o íconos que el usuario pueda asociar a una determinada acción.
- Documentación y ayuda: los sistemas deben tener un lugar de documentación que aclare dudas sobre el producto.
 - Relación entre el sistema y el mundo real: usar los términos que se utilizan en el negocio.

El proceso de diseño de interfaz también es iterativo e incremental. Necesitas retroalimentación constante por parte del usuario, para poder hacer una primera evaluación se utilizan prototipos desechables. Puede que se tengan que hacer varios diseños hasta que el usuario se quede con uno. Es en ese momento cuando empezamos a diseñar prototipos evolutivos. Los prototipos evolutivos sirven para probar la interacción del usuario con el sistema. Una vez que los prototipos han pasado las pruebas de usabilidad allí se genera una versión beta o alfa, que se puede poner en producción.

EVOLUCIÓN DE SOFTWARE

- Sw evoluciona debido al cambio.
- Cambio ocurre cuando → se corrigen errores.
 - el sistema se adapta a un nuevo entorno.
 - el cliente solicita nuevas características/funciones.
- Una vez construido el software y liberada la primera versión de nuestro sistema, hay que empezar a mantener y evolucionar el mismo para que no se vuelva obsoleto.

CICLO DE VIDA EN LA EVOLUCIÓN DEL SW



Identificación de cambio y proceso de evolución

- Las propuestas de cambio son el motor para la evolución de un sistema.
- Cambios provienen de: reqs que no se hayan implementado, peticiones de nuevos reqs, reportes de bugs, y nuevas ideas para la mejora del sw.
- Los procesos de identificación de cambios y evolución del sistema son cíclicos y continúan a lo largo del ciclo de vida del producto.

Actividades del proceso de evolución:

Al recibir una petición de cambio, lo primero que se hace es evaluar el impacto que tendrá el mismo en el sistema, luego se planifican las versiones, lo cual implica reparar los defectos, adaptar la plataforma y perfeccionar el sistema, de ahí se pasa a implementar estos cambios en el sistema, donde podría surgir un nuevo inconveniente el cual se debe resolver, por último, se realiza la entrega del sistema, estando siempre listo para recibir peticiones de cambio

- 1) Análisis de impacto: analizar lo que implica ese cambio a nivel de esfuerzo que va a llevar, cuántos componentes va a afectar y qué hay que cambiar.
- 2) Planificación de versiones: consiste en 3 subetapas: reparación de fallas, adaptación y perfeccionamiento del sistema (nva funcionalidad).
- 3) Implementación de cambios: se deben llevar a cabo los wf de análisis, diseño, implementación y prueba.
- 4) Liberación del sistema a los clientes: se lanza una nva versión del sistema.

El costo de impacto de dichos cambios se valoran para saber qué tanto resultará afectado el sistema por el cambio y cuánto costaría implementarlo.

Si los cambios propuestos se aceptan, se planea una nva versión del sistema. Durante la planeación de la versión se consideran todos los cambios propuestos.

Entonces, se toma una decisión de cuáles cambios implementar en la siguiente versión del sistema, según las prioridades definidas.

Después de implementarse se valida y se libera una nva versión del sistema. Luego el proceso se repite con un conjunto nvo de cambios propuestos para la siguiente versión.

ESTRATEGIAS DE EVOLUCIÓN DEL SOFTWARE

Reingeniería

- Significa reestructurar o reescribir el código de un sw antiguo, una vez que ya fue mantenido. Es el proceso de rediseñar un sistema existente para mejorar su estructura y entendimiento, adaptándolo a nuevas necesidades. Suele implicar documentar nuevamente el sistema, refactorizar su arquitectura, traducción a un nuevo lenguaje o modificar y actualizar la estructura y valores de los datos.
- Es el proceso de analizar, modificar y reconstruir un sistema existente para mejorar su calidad, mantenibilidad o adaptarlo a nuevos requisitos.
- Se crea un nuevo software a partir de uno ya existente.
- Mejorar la calidad interna del producto de software, sin cambiar funcionalidades.
- Busca hacer que el producto sea más fácil de mantener.
- Es el trabajo que se hace sobre el sw existente para mejorar su calidad interna sin modificar la funcionalidad del mismo. El sw hace lo mismo que antes, pero con una calidad mayor.
- Implica una decisión de inversión sobre un producto de sw para mejorar su calidad interna y que quede en una mejor posición para mantenimiento.
- El propósito de hacer reingeniería es la necesidad de mantenimiento del sw pero a un costo menor.

Beneficios:

- Reducción del riesgo: Eliminar y crear un sistema desde cero implica altos riesgos, como errores en la especificación o problemas de desarrollo. La reingeniería minimiza estos riesgos al trabajar sobre una base existente.
- Reducción de costos: El costo de reingeniería puede ser significativamente menor que el costo de desarrollar un sw nuevo.

Refactoring (proceso continuo de mantenimiento)

- Es un proceso de cambio continuo (tenemos variables que cambian continuamente).
- Se utiliza para evitar la degradación del sw y para extenderle la vida útil.
- Es un proceso de modificar un programa para mejorar su estructura, reducir su complejidad o hacerlo más fácil de entender. (para frenar la degradación producida por el cambio).
- Técnica para mejorar el código por dentro sin modificar su estructura por fuera. Se parte de un componente de código que funciona, pero queremos modificarlo para mejorarlo y que sea más eficiente o simplemente que se lo entienda mejor.
- No se agrega funcionalidad.
- Es una herramienta para mejorar o evolucionar la calidad interna de un componente.
- Intención de evitar la degradación de la estructura y el código, que aumentan los costos y las dificultades por mantener el sistema.
- Aspectos que se pueden mejorar mediante refactorización: código duplicado, métodos largos, uso excesivo de switch (case), aglomeración de datos, generalidad especulativa (elementos innecesarios 'por si acaso').

Mantenimiento

- Proceso general de modificar el software una vez entregado, para que siga siendo útil.
- Tipos → *Correctivo*: una vez entregado el sw, éste presenta fallas/errores al momento de utilizarlo. Es no cobrable.
→ *Adaptativo*: cuando algún aspecto del entorno (hw) del sistema cambia el sw. Son modificaciones del RNF. Es cobrable.
→ *Perfectivo*: cuando el cliente desea agregar nvas funcionalidades o cambian las existentes. Son cambios en los RF. Es cobrable.

DISEÑO DE PERSISTENCIA

- Persistencia: capacidad del objeto de permanecer en el tiempo y en el espacio.
- Implica analizar nuestras clases, definir cuáles clases necesitan persistencia, en qué almacenamiento persistente las vamos a guardar, diseñar la BD y la estructura de los datos, y diseñar la persistencia (cómo conectar las clases con la BD).
- El origen de la problemática son los paradigmas, ya que el programa se diseña según el POO, y la BD trabaja con el paradigma relacional o estructurado.
- Incompatibilidad entre la representación de los datos orientada a registros y la orientada a objetos.

Problema de Impedancia

- Ocurre cuando queremos guardar objetos a BD relacionales. El primer problema es que las BDR no soportan comportamiento, sólo guardan datos; y el segundo problema es que no puedo guardar cualquier tipo de datos, porque sólo guardan primitivos
---> no podemos crear tipos de datos
---> predeterminados por la BD --> los recursos que te da el motor de BD.
---> la identidad de los objetos en memoria (por referencia) no coincide con la identidad de las tuplas en la BDR (por clave primaria).
- Este problema se da porque no puedo guardar código en la BD.

- Al trabajar en un lenguaje orientado a objetos, toda la info se almacena en objetos, por lo tanto, se necesita transformar la estructura de info de objetos a una estructura orientada a tablas (modelo relacional).

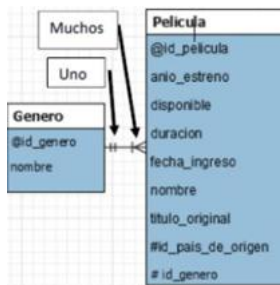
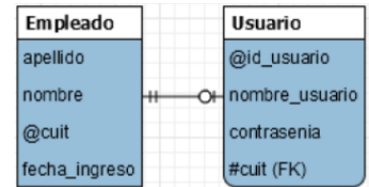
La vinculación entre el modelo de clases y la BDR es el Mapeo o diseño de persistencia.

Las clases persistentes del modelo de dominio se convierte en una tabla y se les descarta el comportamiento. A cada atributo de la clase se lo coloca en una columna de la tabla, y se asigna una clave primaria para identificar de forma única cada fila. La tabla se llama igual que la clase para mantener la trazabilidad o el vínculo.

Mapeo de asociación en base de datos

- Uno a uno: Cada instancia de una tabla se asocia con una instancia de otra tabla.

Usamos claves primarias y foráneas para conectar tablas.

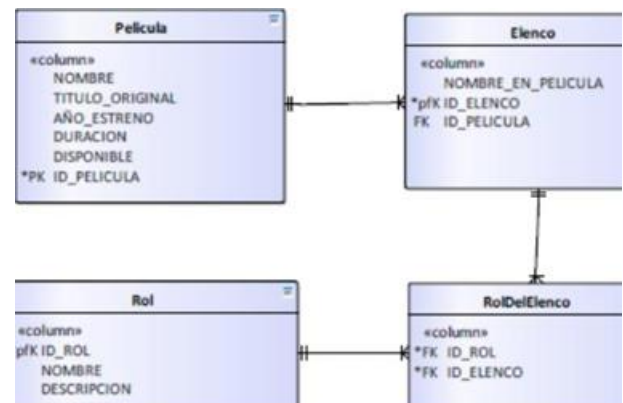


- Uno a muchos o Muchos a uno: 1..* (un género puede estar asociado con varias películas); *.1 (varias películas tienen un solo género).

Se utiliza una clave foránea en la tabla del lado 'muchos' que apunte a la clave primaria del lado 'uno'.

- Muchos a muchos: Ocurre cuando múltiples instancias de una tabla, están relacionadas con múltiples instancias de otra tabla.

Se crea una tabla intermedia para representar la asociación.



RolDelElenco sería la tabla intermedia

Mapeo de herencia

- Jerarquía supertipo-subtipo: Se crean clases separadas para la clase padre y las clases hijas. La tabla padre contiene los atributos comunes; y las tablas hijas tienen una relación uno a uno con la tabla padre y contienen atributos específicos. La clase abstracta está en su propia tabla y las tablas de los hijos hacen referencia a ella. Reduce la redundancia, pero hace JOINS que pueden reducir la performance.
- Eliminación del padre: Los atributos del padre se distribuyen entre las tablas hijas. Se eliminan las relaciones de herencia, pero aumenta el tamaño de las tablas debido a la duplicación de atributos comunes.
- Eliminación de los hijos: Se combinan todos los atributos de las clases hijas con los del padre en una sola tabla. No se recomienda porque hay poca cohesión y hay un posible desperdicio de espacio por atributos no utilizados.

FRAMEWORKS DE PERSISTENCIA (formas de hacer persistencia)

- Sirven para gestionar cómo los objetos en memoria (clases del sistema) se almacenan y recuperan desde una base de datos.
- Es una herramienta o biblioteca que simplifica el proceso de almacenar y recuperar objetos desde una base de datos.
- Un servicio de persistencia tiene que traducir los objetos a registros y guardarlos en una BD, y traducir registros a objetos cuando los recuperamos de la BD.
- Las funciones que deben proporcionar se conocen como materialización y desmaterialización.
- Materializar**: 1 o varias filas de tablas de la BD son convertidas a objetos en memoria.
- Desmaterializar**: un objeto se graba en la BD en forma de 1 o varios registros de tablas.

1) SuperObjetoPersistente

- ❖ Se propone crear una clase de fabricación pura (SuperObjetoPersistente), y en esa clase se debe definir los métodos de materialización y desmaterialización, junto con cualquier método que necesitemos para resolver la persistencia.
- ❖ Todas las clases que necesitan persistencia deberían heredar de la clase SuperObjetoPersistente.
- ❖ Correspondencia directa porque no hay nada en el medio entre la clase de fabricación pura y las clases que necesitan persistencia.
- ❖ El SuperObjetoPersistente define los comportamientos, además de tener la responsabilidad de saber cómo hacerse persistentes en una BD.

Ventajas	Desventajas
No crece la estructura de clases.	Genero fuerte acoplamiento entre la capa de Lógica de Negocio y la capa de BD.
Fácil de implementar.	Rompe el principio de responsabilidad única: las clases de dominio ahora se encargan de saber cómo materializarse y desmaterializarse. También es responsable de la persistencia.
	Rompe el principio open-close: la clase se abre para agregar los métodos materializar y desmaterializar.
	Se rompe la cohesión.

2) Esquema de Persistencia

- ❖ Es una decisión arquitectónica. Agrego una capa (intermedia) de persistencia entre la capa Lógica de Negocio y la BD, compuesto por clases de fabricación pura, que tienen el rol de establecer vínculos entre la capa de Lógica de Negocio y la capa de Datos, sin que los datos se vean afectados por el problema de persistencia.
- ❖ Correspondencia indirecta: enfoque para interactuar entre dos sistemas que no son compatibles de forma directa.
- ❖ Para integrar este esquema a mi modelo de dominio tengo que crear una clase de fabricación pura, por cada clase del dominio que necesite persistencia. Y estas clases manejan las operaciones de materializar y desmaterializar.
- ❖ Le quitamos la responsabilidad a las clases de negocio de saber cómo materializarse y desmaterializarse.

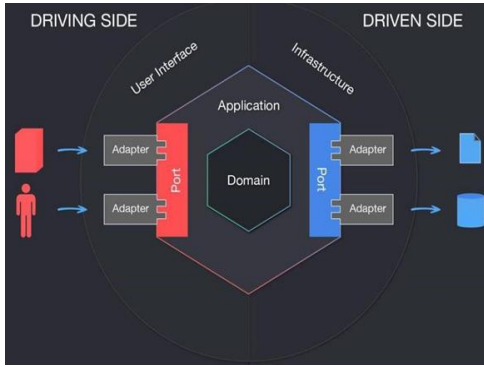
Ventajas	Desventajas
Respeto los principios de responsabilidad única y open-close porque las clases de dominio no se tocan.	Hay un aumento considerable de clases; hace crecer mucho la estructura.
Mantiene alta la cohesión.	
Soporta extensibilidad.	
Mejora la reusabilidad: separa la lógica de negocio de los detalles de persistencia, permitiendo que las clases de negocio se mantengan independientes de la base de datos. Al centralizar la lógica de persistencia en una capa intermedia, se facilita el cambio de mecanismos de almacenamiento sin afectar las clases de negocio.	
Mantiene bajo el acoplamiento, ya que aplica el principio de inversión de dependencia. El control lo toman las clases del esquema de persistencia, que van y buscan a los objetos, y los materializan y desmaterializan.	

ANÁLISIS DE SI / WF DE ANÁLISIS

- Encontrar y describir los objetos del dominio del problema.
 - Resultado/salida => Modelo de análisis **
- * * → describe el dominio desde la perspectiva de la clasificación de objetos.
- es una representación mínima del sistema que modela, suficiente para capturar la lógica esencial del sistema sin entrar en temas de rendimiento o construcción.
- es una primera aproximación al diseño y permite ver el sistema sin tener los detalles de la implementación, lo cual hace que sea más fácil entender el sistema y tener un primer pantallazo de lo que hace el mismo.
- a diferencia del modelo de diseño, el modelo de análisis no resuelve todos los requerimientos y es genérico con respecto a la implementación.
- Entrada: modelo de requerimientos.
 - En el wf de reqs uno determinaba los reqs, pero en el análisis es donde se especifican esos reqs y empezamos a abrir los casos de uso y no solamente ver lo que hacen, sino también ver cómo lo hacen.

ARQUITECTURA HEXAGONAL

Es un diseño que separa la lógica de negocio central de los detalles externos mediante puertos (interfaces) y adaptadores (implementación). Representada como un hexágono, prioriza la independencia tecnológica y la mantenibilidad. Permite que la entrada de usuarios/sistemas externos llegue en un Puerto a través de un Adaptador, y permite que la salida se envíe desde la Aplicación a través de un Puerto a un Adaptador.



- **Puertos:** punto de entrada independiente de la tecnología, determina la interfaz que permitirá a los actores externos comunicarse con la Aplicación, independientemente de qué o quién implementará dicha interfaz. Los puertos también permiten que la Aplicación se comunique con sistemas o servicios externos.

Interfaces que conectan el núcleo con sistemas externos, independientes de tecnologías específicas.

- **Adaptadores:** iniciará la interacción con la Aplicación a través de un Puerto, utilizando una tecnología específica. Implementan los Puertos y actúan como puente con tecnologías externas (APIs, BD).
- **Aplicación:** representada por un hexágono que recibe comandos o consultas en los Puertos y envía solicitudes a otros actores (como BDs, también a través de los Puertos).
- **Lógica de negocio (núcleo):** Contiene las reglas y algoritmos centrales, definiendo interfaces (puertos).
- **Sistemas externos:** Interactúan con la aplicación a través de los adaptadores y puertos.

Funcionamiento:

- ➔ Lado conductor: adaptadores que inician la interacción.
- ➔ Lado conducido: adaptadores que responden a solicitudes del núcleo; aquellos a los que la aplicación "impulsa a comportarse".
- ➔ Los puertos actúan como contratos para mantener la independencia entre el núcleo y las implementaciones externas.

Beneficios:

1. **Acoplamiento bajo:** el núcleo es independiente de las tecnologías externas.
2. **Alta cohesión:** la lógica de negocio se mantiene enfocada.
3. **Pruebas mejoradas:** fácil de testear por su modularidad.
4. **Mantenimiento sencillo:** cambios en adaptadores no afectan el núcleo.
5. **Escalabilidad:** integra múltiples sistemas y soporta alto volumen de datos.
6. **Reusabilidad:** adaptadores intercambiables sin alterar el núcleo.

Layered (en capas)	Hexagonal
Centrada en la base de datos.	Centrada en la lógica de negocio.
Fuerte acoplamiento entre capas y difícil de mantener.	Bajo acoplamiento y fácil integración con nuevas tecnologías.
Menos flexible ante cambios tecnológicos.	Adaptable y escalable.
Organiza el sistema en capas separadas (presentación, lógica de negocio, acceso a datos). Es fácil de mantener en sistemas pequeños debido a su modularidad y la posibilidad de realizar pruebas por capa. Sin embargo, al crecer, los cambios en una capa pueden afectar a las demás, volviendo el sistema más rígido y difícil de modificar.	Se enfoca en desacoplar el núcleo del sistema de las interfaces externas, lo que facilita integrar nuevas tecnologías y realizar cambios sin afectar la lógica de negocio. Aunque es muy mantenible y flexible, requiere un diseño más complejo al principio y puede tener una sobrecarga de gestión debido a la necesidad de adaptadores y puertos.

Se puede aplicar en microservicios y en monolíticos. Plantea que en el centro del hexágono tenés el core de tu negocio, las funcionalidades centrales que realiza tu sistema.

Y después tenés alrededor, una serie de capas que van a estar rodeadas por adaptadores y por endpoints. Tenés un endpoint y diseñás adaptadores para conectarte con el centro.

Mediante un camino que vos establecés, es la forma en la que vas a interactuar con el centro. Es como para proteger la funcionalidad central y darle una capa de seguridad extra.

Entonces, el de afuera va a tener q diseñar una serie de adaptadores para conectarse con mis adaptadores que yo expongo, que se van a comunicar con el centro. Y la relación siempre es de afuera hacia adentro.. se hace inversión de dependencia, siempre va de afuera hacia el centro de tu negocio.

Ej: que la BD se adapte a tu sistema, en vez de que el sistema se amolde a la base de datos. Lo de afuera se amolda a lo interno.

PATRONES GRASP

- Ayudan a construir clases más cohesivas y con el menor grado de dependencia hacia otras clases (acoplamiento).
- Solucionan el problema de qué responsabilidades asignarle a una clase determinada (responsabilidades 'hacer' y 'conocer').
- Hacer: → Realizar acciones propias, como crear objetos, realizar cálculos o responder solicitudes.
 - Delegar acciones a otros objetos.
 - Coordinar actividades entre varios objetos.
- Conocer: → Conocer datos privados encapsulados.
 - Conocer objetos relacionados (atributos o clases vinculadas).
 - Conocer datos que se pueden derivar o calcular.

1. Creador

La clase A debe tener la responsabilidad de crear a la clase B si:

- A contiene a B.
- A tiene la información necesaria para crear a B.
- A conoce a B.

2. Experto en Información

La responsabilidad debe ser asignada a la clase que tiene la información necesaria para realizar esa acción.

3. Alta Cohesión

La responsabilidad a una clase debe asignarse de tal manera que se mantenga alta la cohesión.

Ej: Tengo una clase película y una clase gestor, y quiero averiguar si una película ya se estrenó. Una solución es que el gestor le pida la fecha de estreno a la película y que el gestor verifique si ya se estrenó, pero de esta

manera no mantenemos alta la cohesión porque le estás asignando una responsabilidad al gestor que podría hacer la clase película, ya que tiene la información para hacerlo. La solución más adecuada es que sea la propia película quien tenga la responsabilidad de verificar si ya fue estrenada o no, y le devuelva true o false al gestor.

4. Bajo Acoplamiento

La responsabilidad a una clase debe asignarse de tal manera que se mantenga bajo el acoplamiento. (no crear dependencias innecesarias).

Ej: Tengo una clase torneo y una clase fecha de torneo. ¿Quién sería más conveniente que tenga la responsabilidad de crear la fecha de torneo, el gestor o el torneo? La solución más adecuada es que sea el torneo el encargado de crear la fecha del mismo, ya que contiene la información necesaria para poder hacerlo, además de que seguramente ya existe una relación estructural entre esas dos clases (cosa que con el gestor no pasa).

5. Controlador

Qué clase debe ser responsable de atender un evento del sistema. Para esto se crea una clase controlador que se encarga de manejar o controlar las distintas actividades que se deben realizar.

El controlador deberá tener la responsabilidad de coordinar el caso de uso y es quien efectivamente sabe cuáles son los pasos a seguir, y lo que hace es delegar a los distintos objetos el trabajo que se tiene que hacer.

Asignar la responsabilidad de recibir o manejar mensajes de eventos de un sistema, a una clase controladora de esos eventos.

WF DE DISEÑO

- Entradas: requerimientos del wf de reqs y el modelo de análisis del wf de análisis.
- En el diseño construimos un modelo físico.
- Resolvemos los requerimientos no funcionales.
- Aquí se diseña la arquitectura.
- No modelamos la solución en términos lógicos, sino en términos físicos; se ocupa de la solución de todos los aspectos que tienen que ver con la implementación.
- Salidas:
 - *Modelo de Diseño*: modelo de objetos que describe la realización física de los casos de uso, centrando la atención en como los RF y los RNF tienen impacto en el sistema.
 - *Modelo de Despliegue*: modelo de objetos que describe la distribución física del sistema en cuanto a la distribución de la funcionalidad en los nodos de cómputo. Representa una correspondencia entre la arquitectura del sw y la del hw. Es la descripción de la arquitectura.

¿Qué se diseña?

- Arquitectura: Es el conjunto de decisiones significativas respecto a resolver reqs no funcionales y funcionales. Toma los reqs no funcionales significativos para la arquitectura y los aplica al modelo de análisis.
- Procesos: Transforma elementos estructurales de la arquitectura del programa en una descripción procedimental de los componentes de sw. Se utilizan patrones de diseño para la creación de procesos. Cómo vas a hacer para distribuir los componentes de sw en el hw para cumplir con los RF y RNF del sistema.
- Datos: Implica analizar nuestras clases y definir cuáles de esas clases necesitan persistencia. Transforma los reqs en las estructuras de datos necesarias para hacer persistente el sw.
- IHM: Se encarga de evaluar todos los aspectos que tienen que ver en cómo interactúan las personas con el sw. Esta disciplina combina las interfaces de usuario y la experiencia de usuario.
- Entrada/Salida: Describe cómo se ingresa información al sw y cómo se presentarán las salidas del mismo.
 - En lotes: se acumulan las transacciones para procesarlas después.
 - En línea: en el momento en que ocurre la transacción, ésta es procesada.

- En tiempo real: son sistemas en línea, pero pueden modificar el ambiente donde está inmerso, tienen un tiempo estricto de respuesta.
- **Procedimientos manuales:** Describe cómo se integra el sw al sistema de negocio. Incluye los 'plan B' en caso de que el mismo falle. Nos permite insertar el sw en el negocio.

PATRONES ARQUITECTÓNICOS

- Es una descripción abstracta de buena práctica, que se ensayó y se puso a prueba en diferentes sistemas y entornos. Debe describir cuando es y no es adecuado usarlo, así como sus fortalezas y debilidades.
- Un framework arquitectónico es el conjunto de patrones arquitectónicos que se utilizan para estructurar una aplicación.

Layered (en capas)

- ✚ Consiste en una pila de capas en donde cada una actúa como una máquina virtual de la capa de arriba. Las capas sólo pueden usar la capa que está directamente debajo de ellas.
- ✚ La funcionalidad del sistema está organizada en capas separadas, y cada capa se apoya solo en las facilidades que le ofrece la capa inmediata inferior.
- ✚ El propósito es lograr el mejor nivel de cohesión y acoplamiento posible, decidiendo en qué capa voy a ubicar cada componente de software.
- ✚ Una capa es el nombre que se le da desde la arquitectura a un subsistema. Dentro de una capa hay componentes.

Publish & Suscribe

- ✚ Consiste en componentes independientes que publican eventos y otros que se suscriben a ellos.
- ✚ Los publicantes ignoran el motivo por el cual el evento es publicado, los suscriptores ignoran el por qué o quién publica el evento, y dependen sólo del evento.
- ✚ Cada tópico puede tener más de un publicante, y los publicantes pueden aparecer y desaparecer dinámicamente.
- ✚ Los suscriptores pueden suscribirse y desuscribirse dinámicamente de un tópico.
- ✚ Las arquitecturas basadas en este patrón son muy flexibles y adecuadas para aplicaciones que requieren mensajes asíncronos. Al ser asíncrono no se pierde información si hay pérdida de conexión, la información queda guardada en el tópico y luego se rehace el intento de entregar los mensajes tantas veces como esté configurado.

Messaging

- ✚ Esta arquitectura desacopla emisores de receptores utilizando una cola de mensajes intermedia.
- ✚ La cola es el componente que acumula peticiones que van a ser entregadas en distintos componentes de sw que necesitan recibir esa información.
- ✚ El emisor envía un mensaje al receptor y sabe que eventualmente será entregado, aunque la red esté caída o el receptor no esté disponible.
- ✚ Provee una solución resistente para aplicaciones en las cuales la conexión es transitoria, debido a la poca confiabilidad de la red y de los servidores. En este caso, los mensajes se mantienen en la cola hasta que el servidor se conecta y los elimina. Es apropiada cuando el cliente no necesita respuesta inmediata después de enviar el mensaje.
- ✚ Se usa cuando no se quiere perder información, ya que está pensado para comunicaciones asíncronas, si se produce una interrupción, la información queda acumulada como petición en la cola, no se pierde y cuando se reestablece la conexión se efectúa la entrega de peticiones.

Broker

- ✚ Actúa como concentrador de mensajes, y los remitentes y receptores se conectan como radios. Las conexiones al bróker son por medio de puertos asociados a un formato específico de mensaje.
- ✚ El agente incorpora la lógica de procesamiento para entregar un mensaje recibido en un puerto de entrada a un puerto de salida.

- ✚ La lógica del bróker es que transforma el tipo de mensaje de origen recibido en un puerto de entrada, en el tipo de mensaje de destino en el puerto de salida.
- ✚ Los agentes son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino.
- ✚ Se utiliza cuando hay un problema de compatibilidad, y la mayoría de las veces, se encuentra involucrado alguien ajeno a nuestro sistema que no podemos controlar.

Process Coordinator

- ✚ Encapsula los pasos necesarios para completar un proceso de negocio.
- ✚ Recibe una solicitud de inicio de proceso, llama a los servidores en el orden indicado por el proceso y emite los resultados.
- ✚ Los servidores simplemente definen un conjunto de servicios que pueden ejecutar, y el coordinador los llama cuando es necesario como parte del proceso de negocio.
- ✚ Es utilizado comúnmente para implementar procesos de negocio complejos que deben hacer peticiones a componentes de servidores diferentes. Encapsulando la lógica de proceso en un lugar es mucho más fácil cambiar, administrar y monitorear la performance del proceso.

Client-Server

- ✚ La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usen esos servicios.
- ✚ Los clientes conocen a los servidores y deben saber cómo usarlos, pero los servidores no necesitan conocer a los clientes.
- ✚ Clientes y servidores son procesos lógicos.
- ✚ Los clientes inician la comunicación, hacen peticiones y esperan la respuesta.
- ✚ Los servidores no conocen la identidad de los clientes hasta que se han conectado. Los servidores atienden las peticiones.
- ✚ Ej: colas de impresión. Word es el cliente, y la cola de impresión de Windows es el servidor.

Pipe & Filter

- ✚ Los datos fluyen de un componente a otro (filtros) a través de una tubería, y se transforman conforme se desplazan en la secuencia. Cada paso de procesamiento se implementa como un transformador. Los datos de entrada fluyen por medio de dichos transformadores hasta que se convierten en salida.
- ✚ Se suele utilizar en aplicaciones de procesamiento de datos, donde las entradas se procesan en etapas separadas para generar salidas relacionadas.

Model View Controller (MVC)

- ✚ Separa presentación de interacción de los datos del sistema.
- ✚ El modelo maneja los datos del sistema y las operaciones asociadas a esos datos.
- ✚ La vista define y gestiona cómo se presentarán esos datos al usuario.
- ✚ El controlador dirige la interacción del usuario y pasa esas interacciones a vista y modelo, es decir, recibe peticiones de la vista sobre el modelo, interacciona con el modelo y devuelve el resultado a la vista.
- ✚ Se usa cuando existen múltiples formas de interactuar con los resultados. También cuando se desconocen los requerimientos futuros de la interacción y presentación.

ARQUITECTURA MONOLÍTICA VS MICROSERVICIOS

Arquitectura Monolítica

Una aplicación empaquetada como una sola unidad donde todos los módulos están interconectados. Es autosuficiente, se corre el sistema como un todo, no significa que no se deban respetar los principios de diseño y que los componentes no tengan que ser altamente cohesivos y débilmente acoplados. Se suele pensar que se trata de un estilo antipatrón, pero no es necesariamente así, lo único que significa es que el sistema se compila/ejecuta como si fuera un único componente, es decir, o se ejecuta todo o no se ejecuta nada.

Cabe mencionar que cualquier librería que se requiera será exportada como parte del archivo compilado de salida.

No soporta escalabilidad, a medida que el sistema va creciendo se hace cada vez más complicado de administrar, si el equipo es grande se complica el desarrollo de una manera eficiente, ya que es complicado dividir las tareas sin entrar en constante conflicto.

Es mucho más sencillo de desarrollar; un ejemplo de sistema que es conveniente que sea monolítico son los sistemas ERP, ya que es conveniente que ese tipo de sistemas se ejecuten como un todo y que sean autosuficientes, y no dependan de la conexión a internet para seguir funcionando, pero todo eso va a depender de los requerimientos no funcionales que tenga el sistema y de la escalabilidad; si ese sistema está pensando para que tenga un crecimiento rápido, la arquitectura monolítica no es la mejor opción.

Ventajas de la arquitectura monolítica

- Fácil de desarrollar → Debido a que solo existe un componente, es muy fácil para un equipo de desarrollo iniciar un nuevo proyecto y poner en producción rápidamente.
- Fácil de escalar → Solo es necesario instalar la aplicación en varios servidores y ponerlo detrás de un balanceador de cargas.
- Pocos puntos de fallo → El hecho de no depender de nadie más mitiga gran parte de los errores de comunicación, red, integraciones, etc. Prácticamente los errores que pueden salir son por algún bug del programador, pero no por factores ajenos.
- Autónomo → Las aplicaciones monolíticas se caracterizan por ser totalmente autónomas, lo que les permite funcionar independientemente del resto de aplicaciones.
- Performance → Las aplicaciones monolíticas son significativamente más rápidas debido a que todo el procesamiento lo realizan localmente y no requieren consumir procesos distribuidos para completar una tarea.
- Fácil de probar → Debido a que es una sola unidad de código, toda la funcionalidad está disponible desde el inicio de la aplicación, por lo que es posible realizar todas las pruebas necesarias sin depender de nada más.

Desventajas

- Anclado a un stack tecnológico → Debido a que todo el software es una sola pieza, implica que utilicemos el mismo stack tecnológico para absolutamente todo, lo que impide que aprovechemos todas las tecnologías disponibles. Difícil mitigar a nuevas tecnologías.
- Escalado monolítico → Escalar una aplicación monolítica implica escalar absolutamente toda la aplicación, gastando recursos para escalar funcionalidades que quizás no sean necesarias de escalar.
- El tamaño si importa → Las aplicaciones monolíticas son fáciles de operar con equipos pequeños, pero a medida que la aplicación crece y con ello el equipo de desarrollo, se vuelve cada vez más complicado dividir el trabajo sin afectar funcionalidad que otro miembro del equipo también está moviendo.
- Versión tras versión → Cualquier mínimo cambio en la aplicación implicará realizar una compilación de todo el artefacto y con ello una nueva versión que tendrá que ser administrada.
- Si falla, falla todo → A menos que tengamos alta disponibilidad, si la aplicación monolítica falla, falla todo el sistema, quedando totalmente inoperable.
- Es fácil perder el rumbo → Por la naturaleza de tener todo en un mismo módulo, es fácil caer en malas prácticas de programación, separación de responsabilidades y organización de las clases del código.
- Puede ser abrumador → En proyectos muy grandes, puede ser abrumador para un nuevo programador hacer un cambio en el sistema.

Utilizar aplicaciones monolíticas puede ser una buena opción para aplicaciones que recién comienzan, sin embargo, a medida que la aplicación vaya creciendo es conveniente pensar en migrarla a una arquitectura más modular como la de microservicios.

Arquitectura de Microservicios

Arquitectura basada en componentes pequeños e independientes, cada uno enfocado en una tarea específica y con comunicación vía APIs. Los servicios se conectan a través de interfaces (una API).

El sistema está conformado por componentes independientes que brindan un servicio y que se ejecutan cada uno por separado (posiblemente cada componente tenga su propia base de datos y su propia capa de presentación).

Los microservicios pueden ser un proceso que se ejecuta de forma programada, que muevan archivos de una carpeta a otra, componentes que responden a eventos, etc. Por esto es que, los microservicios no exponen un servicio, sino que brindan un servicio para resolver una determinada tarea del negocio.

Los microservicios son altamente cohesivos porque se enfocan en realizar una única tarea o resolver un único problema. El sistema no se ejecuta como si fuera un único componente, solo se ejecutan los componentes que son necesarios en ese momento, cada componente se ejecuta en el momento en que son llamados o requeridos y cada uno de ellos va a brindar un determinado servicio a través de sus interfaces.

Ayuda a mejorar la escalabilidad de una aplicación, pero disminuye la performance ya que los componentes se encuentran distribuidos. No es autosuficiente. Es útil cuando el sistema es desarrollado por múltiples equipos distribuidos geográficamente; ayuda a trabajar con múltiples tecnologías.

Netflix trabaja con microservicios porque tienen múltiples equipos de trabajo que desarrollan una funcionalidad (servicio) particular y seguramente lo hacen cada uno en una tecnología determinada.

Ventajas de los microservicios

- Alta escalabilidad → Permite montar numerosas instancias del mismo componente y balancear las cargas entre todas las instancias. Se pueden escalar servicios específicos según la demanda (escalabilidad granular).
- Agilidad → Como cada microservicio es un proyecto independiente, cada componente tiene un ciclo de desarrollo diferente del resto permitiendo que se puedan hacer despliegues rápidos en producción sin afectar al resto de componentes.
- Testeabilidad → Los microservicios son fáciles de probar, ya que su funcionalidad es tan reducida que no requiere mucho esfuerzo, además, su naturaleza de brindar servicios hace que sea más fácil crear casos de prueba para probar esos servicios.
- Fácil de desarrollar → Como los microservicios tiene un alcance muy corto, es mucho más fácil para un programador entenderlo, además, cada microservicio puede ser desarrollado por un único programador o por un equipo de trabajo muy reducido.
- Reusabilidad → Como la arquitectura de microservicios se basa en crear componentes que brinden un único servicio (realicen una única tarea o resuelvan un único problema), esto provoca que los componentes sean muy fáciles de reutilizar por otras aplicaciones o microservicios.
- Interoperabilidad → Los microservicios utilizan estándares abiertos para comunicarse, por lo que cualquier aplicación o componente puede comunicarse con ellos, sin importar en qué tecnología se encuentre desarrollado.

Desventajas

- Performance → La naturaleza distribuida de los microservicios agrega una latencia significativa que puede ser un impedimento para aplicaciones donde el performance es lo más importante, por otra parte, la comunicación por la red puede llegar a ser más tardado que el propio proceso en sí. (Comunicación inter-servicios: incrementa la latencia y la necesidad de soluciones robustas).
- Múltiples puntos de falla → La arquitectura distribuida de los microservicios provoca que existan múltiples puntos de falla, pues cada comunicación entre microservicios tiene una posibilidad de fallar, lo cual hay que gestionar adecuadamente. (Pruebas distribuidas: requiere mayor esfuerzo para simular el entorno real).
- Trazabilidad → La naturaleza distribuida de los microservicios complica recuperar y realizar una traza completa de la ejecución de un proceso, porque cada microservicio arroja de forma separada su traza o logs que luego deben ser recopilados y unificados para tener una traza completa.

- Madurez del equipo de desarrollo → Una arquitectura de microservicios debe ser implementada por un equipo maduro de desarrollo y con un tamaño adecuado, pues los microservicios agregan muchos componentes que deben ser administrados, lo que puede ser muy complicado para equipos poco maduros.
- Complejidad del sistema → Difícil coordinar y mantener sistemas distribuidos.
- Gestión de transacciones → Difícil implementar transacciones en múltiples servicios.
- Consumo de recursos → Cada servicio independiente aumenta el uso de memoria y CPU.

Los microservicios se prestan para ser utilizados en aplicaciones que:

- Estén pensadas para tener un gran escalamiento.
- Son tan grandes que es complicado que sean desarrolladas por un equipo de trabajo, lo que hace complicado dividir las tareas sin entrar en constante conflicto.
- Se debe buscar la agilidad en el desarrollo y que cada componente pueda ser desplegado de forma independiente.

Monolítica	Microservicios
Todo el sistema está integrado en una única unidad, lo que facilita el desarrollo inicial y las pruebas. Sin embargo, conforme crece el sistema, los cambios pueden volverse complejos, ya que cualquier modificación afecta a todo el sistema. Esto limita la escalabilidad y la facilidad de mantenimiento a largo plazo.	Dividen el sistema en servicios pequeños e independientes, permitiendo escalabilidad y flexibilidad. Cada servicio puede modificarse sin afectar a los demás, lo que mejora la mantenibilidad. Sin embargo, la gestión de múltiples servicios y la comunicación entre ellos puede ser compleja y costosa, lo que requiere más herramientas y esfuerzo inicial.

Conceptos generales

Acoplamiento: nivel de dependencia entre clases; qué tanto afecta a una clase que otras cambien.

Cohesión: nivel de relación que existe entre los métodos y atributos que tiene una clase. Que una clase sea cohesiva significa que hace pocas cosas y que esas pocas cosas que se hace se encuentran altamente relacionadas.

Delegación: cuando una clase le pasa la responsabilidad a otra de hacer una determinada acción y para ello le pasa todos los datos necesarios para que la pueda ejecutar.

Polimorfismo: Es un concepto fundamental en la programación orientada a objetos que permite que un objeto se comporte de diferentes maneras, dependiendo de su tipo o la clase que lo implemente. Es cuando una o varias clases hija sobrescriben un método de la clase padre para implementar un comportamiento diferente, pero sigue usando la misma interfaz o nombre del método.